

Chapter 5: Database Constraints

Gregory S. DeLozier, PhD

Our pet application has been storing an owner's name with each pet. Now we want to keep information about the owner too. Where does the owner live? What kind of home do they have? Several pets may belong to the same person, so copying those details into every pet record would give us several places to update when that person moves.

We will give owners their own table. Each pet will refer to one owner, and SQLite will help enforce that relationship. The application still uses the database abstraction layer. Its functions now work with related records in two tables.

The example is in `chapter-05-database-constraints/code`. This chapter follows `database.py`, `app.py`, and the templates there. Code excerpts preserve the operations in the example, with line breaks adjusted where needed. Sample records below illustrate the relationship; they do not depend on the contents of the supplied database.

Running the Example

Make a working copy of the code directory for the exercises. With Flask installed in your Python environment, run these commands from that copy:

```
python3 setup_database.py
flask run
```

The setup command prepares `pets.db`. It preserves existing records and adds the `food` column when an older version of this example lacks it. A new database starts with empty tables. This setup is for the owner-and-pet schema used here; use this example's database rather than substituting a database from the earlier single-table application.

Open the pet list at:

```
http://127.0.0.1:5000/list
```

The **Manage Owners** link opens the owner list. On an empty database, create an owner before creating a pet. The pet form needs an owner to offer in its dropdown.

The application calls `database.initialize("pets.db")` at startup. That opens the file and enables foreign-key enforcement. Table creation belongs to the separate setup command. Supplying a filename to setup, such as `python3 setup_database.py practice.db`, prepares that file; it does not change the filename selected by the app.

An Owner Is a Record

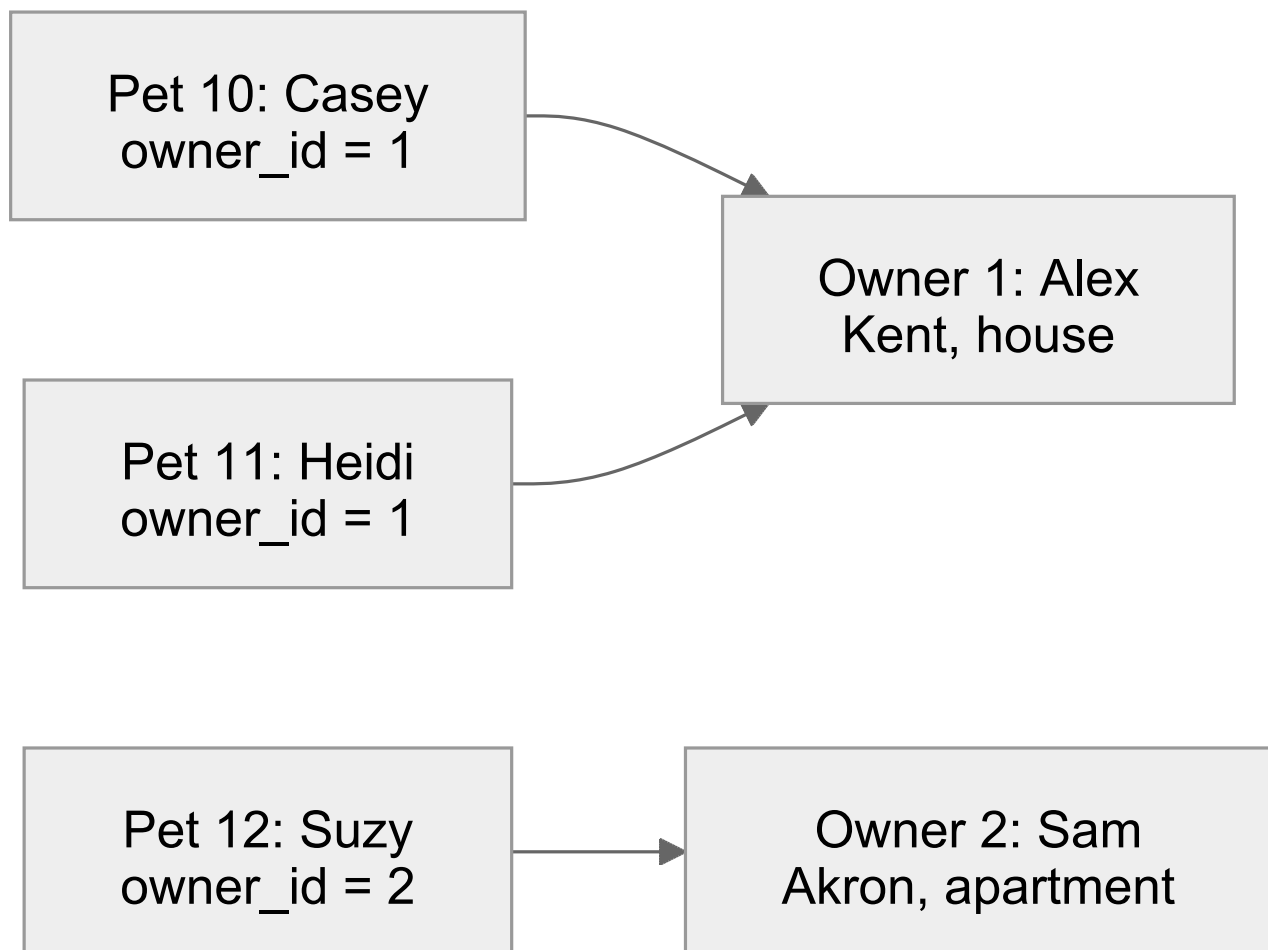
An owner's name is something we can display, but it is not a reliable identifier. Two people can have the same name. A person's name can also change.

The owner table gives each person an integer identifier. The pet stores that identifier in `owner_id`.

owner.id	name	city	type_of_home	
1	Alex	Kent	house	
2	Sam	Akron	apartment	

pet.id	name	type	food	owner_id
10	Casey	dog	kibble	1
11	Heidi	cat	tuna	1
12	Suzy	mouse	seeds	2

Casey and Heidi refer to the same owner record. Changing Alex's city changes one row. Neither pet needs a new `owner_id`.



This is a **one-to-many relationship**: one owner can have many pets. An owner can also have no pets. In this application, each pet must have exactly one owner.

The Two Tables

A **schema** describes the tables and the rules for their data. We write these definitions in Structured Query Language (SQL). The setup function creates the owner table first:

```
CREATE TABLE IF NOT EXISTS owner (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  city TEXT,  
  type_of_home TEXT  
);
```

The **primary key** identifies a row. Our insert statements omit `id`, so SQLite supplies it. The `AUTOINCREMENT` declaration prevents automatic reuse of identifiers from previously committed rows in this table.

The pet table keeps the pet's own information, including food, and adds the reference to its owner:

```
CREATE TABLE IF NOT EXISTS pet (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  type TEXT NOT NULL,  
  age INTEGER,  
  food TEXT,  
  owner_id INTEGER NOT NULL,  
  FOREIGN KEY (owner_id)  
    REFERENCES owner(id) ON DELETE RESTRICT  
);
```

`owner_id` is a **foreign key**. Its value refers to the primary key of an owner row. The column does not copy that owner's name or city. It identifies the record that holds those values.

Rules the Database Enforces

A **constraint** is a rule the database checks when an operation would change its data. These tables use several rules together:

Declaration	Rule in this application
PRIMARY KEY	Each row has its own identifier.
name TEXT NOT NULL	A name cannot be SQL NULL.
type TEXT NOT NULL	A pet's type cannot be SQL NULL.
owner_id INTEGER NOT NULL	Every pet must supply an owner identifier.
REFERENCES owner(id)	That identifier must refer to an existing owner.
ON DELETE RESTRICT	An owner with pets cannot be deleted.

SQL uses **NULL** to represent a missing value. **NOT NULL** rejects that value. An empty string, `' '`, is still a string, so it is different from **NULL**. In the owner form, the application separately checks for a blank name.

The two rules on `owner_id` do different work. **NOT NULL** requires a value. The foreign key requires a matching owner. Supplying 999999 does not satisfy the relationship just because it is an integer.

Keeping references valid is called **referential integrity**. Here, it means that a pet cannot point to an owner record that does not exist.

Enabling Foreign Keys

The connection setup explicitly enables foreign-key enforcement:

```
connection = sqlite3.connect(
    database_file, check_same_thread=False
)
connection.row_factory = sqlite3.Row
connection.execute("PRAGMA foreign_keys = ON")

fk = connection.execute("PRAGMA foreign_keys").fetchone()[0]
assert fk == 1, "Foreign key constraints are not active on this connection."
```

PRAGMA is SQLite's command for settings and other database information. The first command enables the setting. The second reads it back, and the assertion verifies that it is on. The code does this immediately after opening the connection, before any writes.

The setting belongs to the connection. When experimenting in the SQLite command-line program, enable it there too:

```
PRAGMA foreign_keys = ON;
PRAGMA foreign_keys;
```

The second command should return 1. The app also provides `/health`, which reports ok when its own connection has foreign-key enforcement enabled.

Working Through the Abstraction Layer

The database module now has a parallel set of operations for owners and pets:

Operation	Pets	Owners
Read all	<code>get_pets()</code>	<code>get_owners()</code>
Read one	<code>get_pet(id)</code>	<code>get_owner(id)</code>
Create	<code>create_pet(data)</code>	<code>create_owner(data)</code>
Update	<code>update_pet(id, data)</code>	<code>update_owner(id, data)</code>
Delete	<code>delete_pet(id)</code>	<code>delete_owner(id)</code>

The read functions return dictionaries, or lists of dictionaries. A lookup for a missing identifier returns `None`. The create functions return the new row's identifier after committing the insert. Update and delete functions commit their work and return `None`.

Creating an owner inserts that person's details:

```
cursor.execute(
    """insert into owner(name, city, type_of_home)
       values (?, ?, ?)""",
    (data["name"], data.get("city"), data.get("type_of_home")),
)
connection.commit()
return cursor.lastrowid
```

`lastrowid` gives the identifier assigned by the insert. Direct Python code can use that result to create a pet for the new owner. The web application obtains owner identifiers by reading the owner table when it prepares the pet form.

Choosing an Owner in the Form

The route that displays the create form loads the owners:

```
owners = database.get_owners()
return render_template("create.html", owners=owners)
```

The template builds an option for each owner:

```
<select name="owner_id">
  <option value="">-- Select an Owner --</option>
  {% for owner in owners %}
    <option value="{{ owner['id'] }}">{{ owner['name'] }}</option>
  {% endfor %}
</select>
```

The browser displays the owner's name. The option's value carries the identifier. Choosing Alex in the illustrative data submits owner_id as the string "1".

This keeps names convenient for people while the database uses identifiers for the relationship. The form still supplies the pet's name, age, type, and food as separate fields.

Saving a Pet

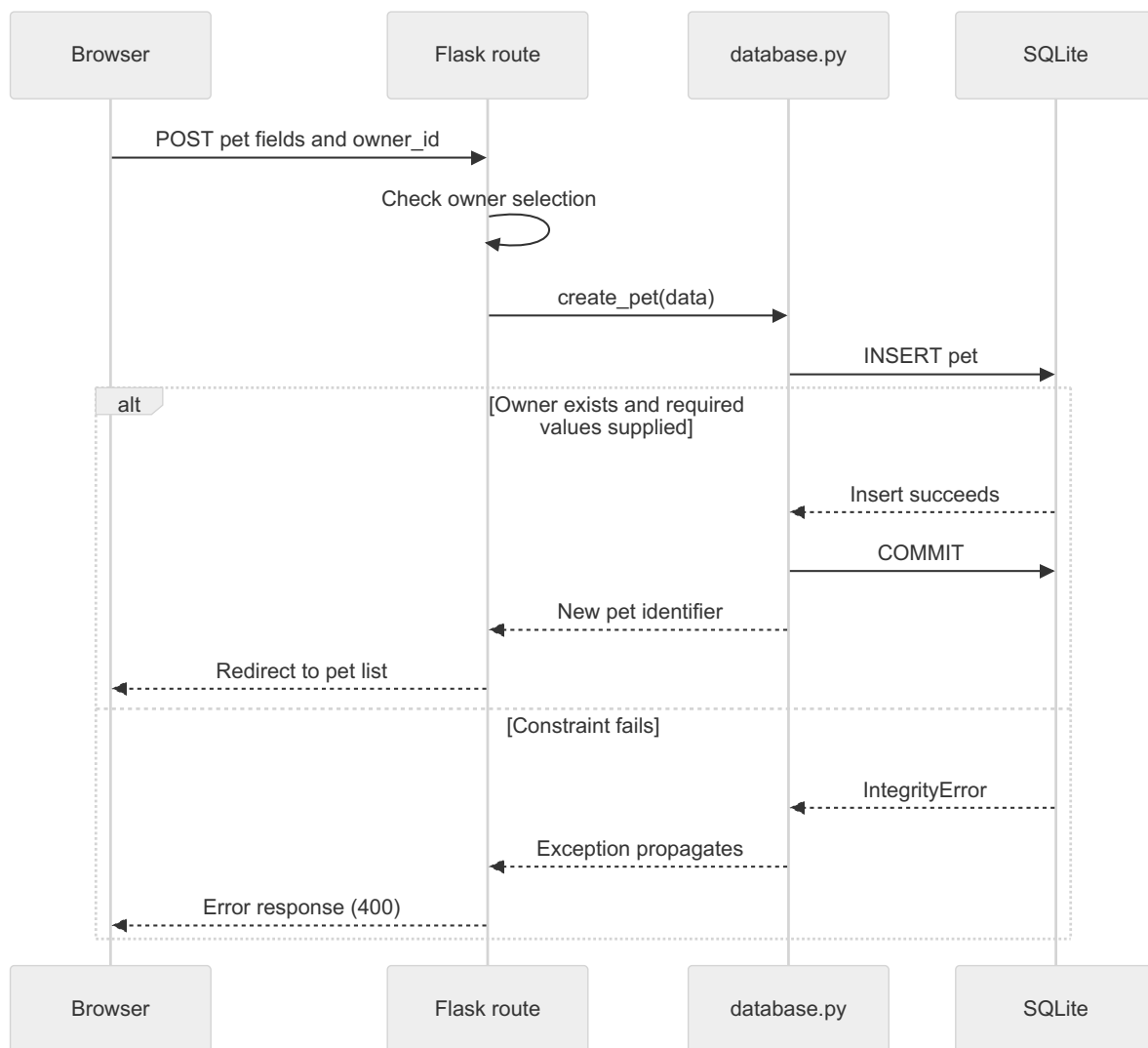
The POST route reads the form into a dictionary. POST is the request method the browser uses here to submit a change. The route first checks whether the owner selection is blank or contains something other than digits:

```
owner_id = (data.get("owner_id") or "").strip()
if owner_id == "":
    return error_page("Error: You must select an owner for the pet.", 400)
if not owner_id.isdigit():
    return error_page("Error: owner_id must be a number.", 400)
```

Then database.create_pet(data) performs the insert. After converting age, it binds the values to the SQL statement:

```
cursor.execute(
    """insert into pet(name, age, type, food, owner_id)
       values (?, ?, ?, ?, ?)""",
    (data["name"], data["age"], data["type"],
     data.get("food", ""), data["owner_id"]),
)
connection.commit()
return cursor.lastrowid
```

The question marks are parameter placeholders. The supplied values follow the column order. SQLite checks the constraints as it carries out the insert. A valid owner reference allows the save to complete.



The route catches `sqlite3.IntegrityError` and returns a readable error response. A failed insert does not reach the following `commit()` or the success redirect.

Application validation and database constraints cooperate. The route can explain a missing selection immediately. The database checks the actual stored relationship, including requests made directly through the Python module.

Displaying the Owner's Name

The pet row holds an identifier. The list page needs a name. The route loads both collections and matches their identifiers:

```

for pet in pets:
    pet["owner_name"] = "<Unknown>"
    for owner in owners:
        if owner["id"] == pet["owner_id"]:
            pet["owner_name"] = owner["name"]
  
```

owner_name is a value added to the dictionary for display. It is not another column in the pet table. The template reads it alongside the pet's other fields.

The route performs the matching each time it prepares the list. After an owner changes their name, the next list request displays that new name for all of their pets. Their identifiers and the references stored in the pet table stay the same.

Updating the Relationship

The edit route loads both the selected pet and the owner list. The template marks the pet's current owner as selected:

```
<option value="{{ owner['id'] }}"
    {% if owner['id'] == data['owner_id'] %}selected{% endif %}>
    {{ owner['name'] }}
</option>
```

Saving the form calls `update_pet(id, data)`. Its statement updates the pet's fields and owner reference together:

```
UPDATE pet
SET name=?, age=?, type=?, food=?, owner_id=?
WHERE id=;
```

For example, suppose Casey is pet 10, currently assigned to Alex, owner 1. Select Sam, owner 2, in the dropdown and save. Using the illustrative records, the route's call is equivalent to:

```
database.update_pet(10, {
    "name": "Casey", "age": "9", "type": "dog",
    "food": "kibble", "owner_id": "2"
})
```

After converting age, the function binds ("Casey", 9, "dog", "kibble", "2", 10) to the statement. The 2 supplies the new owner reference. The final 10 identifies the pet to update. After the commit, Casey still has identifier 10, but owner_id is now 2. The refreshed list displays Sam as Casey's owner. Casey's other fields retain their submitted values, and both owner records remain.

Selecting a different owner changes owner_id. It does not create a new owner or modify the old one. SQLite checks the new reference just as it checks a reference on insert. A nonexistent owner causes the update to fail, leaving the stored pet values unchanged.

Deleting an Owner

What should happen when someone tries to delete Alex while Casey and Heidi still refer to that owner record? Our schema answers with `ON DELETE RESTRICT`: reject the deletion.

The database function attempts an ordinary parameterized delete:

```
cursor.execute("delete from owner where id = ?", (id,))
connection.commit()
```

SQLite raises `sqlite3.IntegrityError` when pets still reference the owner. The Flask route catches that exception and explains why the deletion could not proceed. Both the owner and the pets remain stored.

Removing the relationship first makes the owner eligible for deletion. We can reassign all of that owner's pets to another existing owner. We can also delete the pet records when that is the intended operation. Deleting an owner does not automatically delete their pets in this schema.

Checking the Rules with Code

Run the module's embedded tests from the code directory:

```
python3 database.py
```

The test setup recreates `test_pets.db`, creates owners, and uses their returned identifiers to create pets. It does not recreate the app's `pets.db`. Successful completion prints `done.`

The tests check stored values, including food after an update. They also try operations that should fail. Here is the invalid-owner test:

```
try:
    create_pet({"name": "ghost", "age": 1,
               "type": "dog", "owner_id": 999999})
    assert False, "Expected FOREIGN KEY constraint failure, but insert succeeded."
except sqlite3.IntegrityError as e:
    msg = str(e).lower()
    assert "foreign key" in msg or "constraint" in msg
```

The test deliberately supplies an owner identifier absent from its fresh database. The expected result is an exception. Another test checks that an owner with pets cannot be deleted, then removes the pet and confirms that the owner can be deleted.

These tests call the same database functions as the routes. They demonstrate that the relationship rules operate below the web interface.

Practice: Following an Owner Reference

1. Create an owner and two pets belonging to that owner. Give the pets different foods.
2. Change the owner's name. Reload the pet list and explain why both pets display the change.
3. Try deleting that owner while the pets still belong to them. Check that the records remain.
4. Create another owner and reassign one pet. Try the deletion again. Explain why the remaining pet still matters.

5. Reassign the second pet and delete the original owner. Confirm that both pets remain with their new owner and retain their food values.

Trace one reassignment through the form's `owner_id`, the route, and the `UPDATE` statement. Identify where the application checks the submitted value and where SQLite checks the relationship.

Exercise: Give Food Its Own Table

Food is still text stored in each pet record. Refactor it into a separate `food` table, using the owner relationship as a model. For this exercise, each pet selects one food, and several pets may select the same food.

Start with a food identifier and a name. Decide whether every pet must have a food selection or whether an unknown food is allowed. Write that decision down before choosing whether `pet.food_id` needs `NOT NULL`.

Draw the relationship, then write the proposed table definitions. The pet should refer to `food.id` using a foreign key. Keep its owner relationship intact. Decide what should happen when someone tries to delete a food that pets still use, and explain the choice.

Implement the change in a copy of the application:

1. Add functions for working with food records in `database.py`.
2. Replace the pet's free-text food input with a dropdown that displays names and submits identifiers. The edit form should select the current food.
3. Update pet creation and editing to store `food_id`. Display the food's name on the pet list.
4. Provide a way to manage the food records through the application.
5. Describe how existing food strings would become food records and references. Account for blank or missing values. Implement that conversion on a copy of the database, or begin with a fresh database and submit the conversion plan separately.

Test the relationship directly through the database module as well as through the forms. Two pets should be able to share one food record. Renaming that food should change both displayed names without updating the pet rows. A nonexistent food identifier should fail. Check the deletion rule you chose and confirm that owner assignment still works.

Which parts of the change stayed inside the database layer? Which parts required new interface fields or pages? Explain why changing the data model reaches more of the application than changing how the same interface stores its records.

Questions

1. Why store `owner_id` instead of using an owner's name as the reference?
2. How do `NOT NULL` and a foreign key enforce different rules on the same column?
3. What does the owner dropdown display, and what does it submit?
4. Why can an owner have no pets while a pet cannot have no owner here?
5. How does `ON DELETE RESTRICT` affect deletion? How could an owner be removed while keeping their pets?

6. Why must a separate SQLite command-line connection enable foreign keys too?

Further Reading

- SQLite foreign keys, including enabling enforcement and deletion actions: <https://www.sqlite.org/foreignkeys.html>
- SQLite table definitions and constraints: https://www.sqlite.org/lang_createtable.html
- Wikipedia overview of referential integrity: https://en.wikipedia.org/wiki/Referential_integrity
- W3Schools SQL foreign-key tutorial: https://www.w3schools.com/sql/sql_foreignkey.asp